

Evaluation Metrics and Scoring

تعلم الآلة
دكتور فراس صبار

1. Definition:

Evaluation Metrics and Scoring are methods used to measure how well a machine learning model performs on a given task. They provide numerical values or scores that summarize the model's predictive quality, such as how accurate, precise, or reliable its outputs are. These metrics help compare different models, choose the best one, metrics used to evaluate how accurately a classifier predicts class labels.

2. A confusion matrix

A confusion matrix is a detailed way to evaluate the performance of a binary classification model. It is a 2x2 table where:

- Rows represent the true classes (Actual class),
- Columns represent the predicted classes.

Each cell shows how many samples of a true class were predicted as each class. The diagonal values (top-left and bottom-right) represent correct predictions, while off-diagonal values show classification mistakes.

Then the confusion matrix entries correspond to:

- TP (True Positives): correctly predicted positive samples
- TN (True Negatives): correctly predicted negative samples
- FP (False Positives): negative samples incorrectly predicted as positive
- FN (False Negatives): positive samples incorrectly predicted as negative

		Predicted class		
		yes	no	Total
Actual class	yes	TP	FN	P
	no	FP	TN	N
Total		P'	N'	P + N

Confusion matrix, shown with totals for positive and negative tuples

Example: Confusion matrix for the classes *buys_computer = yes* and *buys_computer = no*, where an entry in row *i* and column *j* shows the number of tuples of class *i* that were labeled by the classifier as class *j*. Ideally, the nondiagonal entries should be zero or close to zero.

Classes	<i>buys_computer = yes</i>	<i>buys_computer = no</i>	Total	Recognition (%)
<i>buys_computer = yes</i>	6954	46	7000	99.34
<i>buys_computer = no</i>	412	2588	3000	86.27
Total	7366	2634	10,000	95.42

3. Evaluation measures for binary classification

These metrics are calculated using the values TP, TN, FP, and FN from the confusion matrix.

- **Accuracy (Recognition Rate):** Measures the overall proportion of correctly classified samples.

$$\text{accuracy} = \frac{TP + TN}{P + N}$$

- **Error Rate (Misclassification Rate):** Percentage of incorrect predictions.

$$\text{error rate} = \frac{FP + FN}{P + N}$$

- **Sensitivity (True Positive Rate, Recall):** Measures how many actual positives were correctly identified.

$$\text{sensitivity} = \frac{TP}{P}$$

$$\text{recall} = \frac{TP}{TP + FN} = \frac{TP}{P}$$

- **Specificity (True Negative Rate):** Measures how many actual negatives were correctly identified.

$$\text{specificity} = \frac{TN}{N}$$

- **Precision:** Measures how many predicted positives were truly positive.

$$\text{precision} = \frac{TP}{TP + FP}$$

- **F-score / F1-score:** The harmonic mean of precision and recall.

$$F = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Measure	Formula
accuracy, recognition rate	$\frac{TP + TN}{P + N}$
error rate, misclassification rate	$\frac{FP + FN}{P + N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP + FP}$
$F, F_1, F\text{-score}$, harmonic mean of precision and recall	$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$

Example :

Actual \ Predicted	yes	no
yes	6954 TP	46 FN
no	412 FP	2588 TN

So we have:

- **TP = 6954**
- **FN = 46**
- **FP = 412**
- **TN = 2588**

And:

- **P = TP + FN = 6954 + 46 = 7000**
- **N = TN + FP = 2588 + 412 = 3000**

1. Accuracy

$$\frac{TP + TN}{P + N}$$

$$\frac{6954 + 2588}{10000} = 0.9542 = 95.42\%$$

2. Error Rate

$$\frac{FP + FN}{P + N}$$

$$\frac{412 + 46}{10000} = 0.0458 = 4.58\%$$

3. Sensitivity / Recall (True Positive Rate)

$$\frac{TP}{P}$$

$$\frac{6954}{7000} = 0.9934 = 99.34\%$$

4. Specificity (True Negative Rate)

$$\frac{TN}{N}$$

$$\frac{2588}{3000} = 0.8627 = 86.27\%$$

5. Precision

$$\frac{TP}{TP + FP}$$

$$\frac{6954}{6954 + 412} = \frac{6954}{7366} \approx 0.9441 = 94.41\%$$

6. F1-score

$$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Where:

- Precision = 0.9441
- Recall = 0.9934

$$F1 = \frac{2 \times 0.9441 \times 0.9934}{0.9441 + 0.9934}$$

$$= \frac{1.876}{1.9375} = 0.969 \approx 96.9\%$$

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
import numpy as np

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Train model
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
```

جزء برمجة المصنف

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# Positive and Negative counts
P = TP + FN
N = TN + FP

# Accuracy
accuracy = (TP + TN) / (P + N)

# Error Rate
error_rate = (FP + FN) / (P + N)

# Precision
precision = TP / (TP + FP)

# Recall (Sensitivity)
recall = TP / (TP + FN)

# Specificity
specificity = TN / (TN + FP)

# F1-score
f1 = 2 * (precision * recall) / (precision + recall)
```

TP FN
FP TN

جزء برمجة
Confusion Matrix

جزء طباعة النتائج

```
print("Confusion Matrix:")
print(cm)

print(f"\nAccuracy : {accuracy:.4f}")
print(f"Error Rate : {error_rate:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"Specificity : {specificity:.4f}")
print(f"F1-score : {f1:.4f}")
```


4. The holdout method

The **holdout method** splits data into a training set and a test set (usually 2/3 training and 1/3 testing). The model is trained on one part and evaluated on the other, but the accuracy may be slightly pessimistic.

Random subsampling repeats the holdout process **k times** with different random splits, and the final accuracy is the **average** across all runs for a more reliable estimate.

5. cross-validation

100 sample / 10 50 each 10 sample

In **k-fold cross-validation**, the dataset is split into **k equal parts**. Each part is used once as a test set while the remaining **k-1 parts** train the model. This process repeats **k times**, and the final accuracy is based on all iterations. Every sample is used for training and testing exactly once.

Leave-one-out cross-validation is the special case where **k equals the number of samples**, meaning each test set contains only one tuple—useful for very small datasets.

In **stratified cross-validation**, each fold preserves the same class distribution as the original dataset. In practice, **stratified 10-fold cross-validation** is recommended because it achieves good accuracy with low bias and variance.

6. Bootstrap method

In the **bootstrap method**, training data is created by sampling with **replacement**, meaning the same tuple can appear multiple times in the training set. One popular version is the **.632 bootstrap**:

- From a dataset of **d tuples**, we randomly sample **d times with replacement** to form a training set.
- Any tuples **not selected** become the test set.
- On average, **63.2%** of the original tuples appear in the training set, and **36.8%** form the test set.

This percentage comes from the probability that a tuple is never chosen:

$$(1 - \frac{1}{d})^d \approx e^{-1} = 0.368$$

The process can be repeated **k times**, and the final accuracy is the average of all bootstrap estimates.

```
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

# Load dataset
iris = load_iris()

# Create model
logreg = LogisticRegression(max_iter=5000)

# Perform 5-fold cross-validation
scores = cross_val_score(logreg, iris.data, iris.target, cv=5)

print("Cross-validation scores:", scores)
print("Mean Accuracy:", scores.mean())
```

Cross-validation

Handwritten signature